Single Cycle RISCV 32 Bits Microprocessor

Draft

Contents

List of Figures

Draft

Glossary

- **MSB (Most Significant Bit)** — The bit in a binary word with the highest positional weight. In a 32-bit RISC-V datapath, bit 31 is the MSB and determines the sign in signed operations.
- **LSB (Least Significant Bit)** — The bit in a binary word with the lowest positional weight. In a 32-bit architecture, bit 0 is the LSB and determines alignment and byte selection in memory operations.
- **RV32I** — The 32-bit base integer instruction set architecture (ISA) of RISC-V. It defines arithmetic, logical, branch, load/store, and control instructions implemented in this microprocessor.
- **Instruction Word** — A 32-bit encoded value fetched from instruction memory that contains opcode, register addresses, funct3, funct7, and immediate fields.
- **Opcode** — A field within the instruction word that determines the instruction type (R-type, I-type, S-type, B-type, U-type, J-type) and drives the control unit decoding logic.
- **Funct3 / Funct7** — Subfields of the instruction used to refine the operation type inside a given opcode group (e.g., ADD vs SUB differentiation).
- **Datapath** — The structural hardware path that processes data, including the Program Counter (PC), Register File, ALU, Immediate Generator, and memory interfaces.
- **Control Unit (CU)** — The combinational logic block responsible for decoding the instruction word and generating control signals such as ALU control, register write enable, memory write enable, and branch selection.
- **ALU (Arithmetic Logic Unit)** — The combinational block that performs arithmetic and logical operations (ADD, SUB, AND, OR, XOR, shifts, comparisons) based on control signals.
- **Register File (PRF)** — A bank of 32 general-purpose registers (x0–x31) used for operand storage and write-back operations. Register x0 is hardwired to zero.
- **Program Counter (PC)** — A register that holds the address of the current instruction. In a single-cycle architecture, PC is updated every clock cycle.
- **Immediate Generator (ImmGen)** — A combinational block that extracts and sign-extends immediate fields from the instruction word depending on the instruction format.
- **Branch Condition (BCOND)** — The logical comparison result used to determine whether a branch instruction modifies the PC.
- **Write-Back (WB)** — The stage in which the ALU or memory result is written into the destination register.
- **Single-Cycle Architecture** — A processor implementation in which each instruction completes in exactly one clock cycle.
- **Interface (SystemVerilog)** — A construct used in verification to group related signals and simplify DUT-testbench connectivity.
- **SVA (SystemVerilog Assertions)** — Formal properties used to verify protocol correctness, signal relationships, and architectural constraints (e.g., PC alignment, register write behavior).
- **Coverage** — Verification metric used to measure how thoroughly the instruction space and control paths have been exercised.

1 Introduction

This document presents the verification methodology and architectural overview of a custom-designed RV32I single-cycle RISC-V microprocessor. The primary objective of this project is to validate the functional correctness, control behavior, and datapath integrity of the processor at the RTL level using structured verification techniques.

The verification environment is built around a SystemVerilog-based testbench that applies controlled stimuli to the RTL design. Stimulus generation and signal observation are performed through a dedicated verification interface, enabling clean separation between the Design Under Test (DUT) and the testbench components.

The verification strategy includes:

- Directed and constrained instruction stimulus applied to the RTL processor.
- Signal abstraction and connectivity using SystemVerilog interfaces.
- Non-intrusive property checking using `bind` constructs.
- Formal and concurrent assertions (SystemVerilog Assertions, SVA) to validate architectural constraints and protocol correctness.
- Functional coverage using cover properties and covergroups to measure exercised instruction space and control paths.

The processor under verification implements the RV32I base integer instruction set architecture in a single-cycle configuration. Each instruction completes within one clock cycle, requiring precise coordination between the control unit, datapath elements, register file, ALU, immediate generator, and memory interfaces.

The following sections describe the architectural organization of the microprocessor, its RTL structure, verification approach, and assertion-based validation framework.

2 RV32I Single-Cycle Microprocessor

2.1 Overview

The `microprocessor_top` module represents the top-level integration of a custom-designed RV32I single-cycle RISC-V microprocessor. This module structurally connects all major architectural blocks required to execute one instruction per clock cycle.

The processor implements the base RV32I instruction set and integrates the following components:

- Program Counter (PC) and next-PC selection logic.
- Instruction Memory for instruction fetch.
- Register File (32 general-purpose registers).
- Immediate Generator.
- Arithmetic Logic Unit (ALU).

- Branch comparator and branch decision logic.
- Data Memory for load and store operations.
- Write-back selection multiplexer.
- Control Unit for instruction decoding and control signal generation.

The design follows a structural RTL approach, where this top-level module primarily consists of signal interconnections and submodule instantiations. All state elements are contained within dedicated submodules such as the program counter, register file, and memory blocks.

For verification purposes, the module also allows instruction injection through an external stimulus mechanism, enabling controlled execution scenarios during testbench-driven validation.

2.2 Microprocessor - Port Conventions

Port	Bits Used	Type	Description
`clk`	1	Input	Global synchronous clock driving all sequential elements of the processor.
`arst_n`	1	Input	Asynchronous active-low reset signal used to initialize state elements.
`instr_en`	1	Input	Instruction override enable. When asserted, the processor executes the externally driven instruction instead of the instruction fetched from memory.
`driven_instr`	[31:0]	Input	External instruction provided by the verification environment for controlled stimulus injection.

2.3 RTL Snippet

```
1 `timescale 1ns / 1ps
2 import riscv_params_pkg::*;
3 //
      ----------------------------------------------------------------------------------

4 // Module: microprocessor_top
5 // Description:
6 //   Top-level integration for a single-cycle RV32-style microprocessor
      datapath.
7 //   Contains:
8 //     - PC register and next-PC selection (sequential vs redirect)
9 //     - Instruction fetch (instruction memory)
10 //     - Register file read/write-back
11 //     - Immediate generation
12 //     - ALU execute
13 //     - Data memory (stores/loads when implemented)
14 //     - Control unit decode and branch decision
15 //
16 // Notes:
17 //   - Signal naming follows datapath intent: pc_* , if_* , id_* , ex_* , mem_
      * , wb_*
18 //   - This module is structural (wires + instantiations) and contains no
      state
```

```
//     besides submodules (PC, RF, memories).
//
    ------------------------------------------------------------------------------

module microprocessor_top (
    input logic                     clk,
    input logic                     arst_n,
    input logic                     instr_en,
    input logic [DATA_WIDTH-1:0]    driven_instr
);


    //
        ------------------------------------------------------------------------

    // IF: Program Counter and Instruction Fetch
    //
        ------------------------------------------------------------------------


    logic [DATA_WIDTH-1:0] pc_q;            // current PC (byte address)
    logic [DATA_WIDTH-1:0] pc_next;         // next PC selected by pc_next mux
    logic [DATA_WIDTH-1:0] pc_plus_inc;     // sequential next PC (PC + inc)
    logic                  pc_inc_sel;      // +4 vs +2 selector (future)
    logic [2:0]            pc_inc;          // increment constant (2 or 4)

    logic [DATA_WIDTH-1:0] instruction;
    logic [DATA_WIDTH-1:0] mem_instruction;           // fetched instruction
                                    Draft
    // Next-PC select:
    //   - branch_taken=0 -> sequential PC+4
    //   - branch_taken=1 -> redirect target (computed in EX path)
    logic                  branch_taken;
    logic [DATA_WIDTH-1:0] branch_target;   // target address (PC + imm),
        produced in EX

    // PC next mux: select sequential or redirect target
    mux #(.WIDTH(DATA_WIDTH)) upc_next_mux_i (
        .in1(branch_target),     // sel=1 -> redirect
        .in2(pc_plus_inc),       // sel=0 -> sequential
        .sel(branch_taken),
        .out(pc_next)
    );

    // PC register
    program_counter pc_i (
        .clk   (clk),
        .arst_n(arst_n),
        .pc_in (pc_next),
        .pc_out(pc_q)
    );

    // PC increment select (+4 now; +2 reserved for future compressed support)
    plus_4_or_2_mux pc_inc_sel_i (
        .sel            (pc_inc_sel),
        .instruction_add(pc_inc)
```

```
68      );
69
70      // PC + increment
71      adder #(.DATA_WIDTH(DATA_WIDTH)) pc_adder_i (
72          .constant_operand(pc_inc),
73          .instruction_addr(pc_q),
74          .next_instruction (pc_plus_inc)
75      );
76
77      // Instruction memory (word-indexed; PC is byte-addressed)
78      instruction_memory #(.DATA_WIDTH(DATA_WIDTH), .DEPTH(DEPTH)) instr_mem_i (
79          .pc   (pc_q),
80          .instr(mem_instruction)
81      );
82
83      //
            ------------------------------------------------------------------------
84      // ID: Decode and Register File Read
85      //
            ------------------------------------------------------------------------
86
87      logic                  rf_we;
88      logic [DATA_WIDTH-1:0] rs1_data;
89      logic [DATA_WIDTH-1:0] rs2_data;
90
91      // Write-back data (selected later)
92      logic [DATA_WIDTH-1:0] wb_data;
93
94      physical_register_file #(.DIR_WIDTH(DIR_WIDTH), .DATA_WIDTH(DATA_WIDTH))
            prf_i (
95          .clk       (clk),
96          .arst_n    (arst_n),
97          .write_en  (rf_we),
98          .read_dir1 (instruction[19:15]),   // rs1
99          .read_dir2 (instruction[24:20]),   // rs2
100         .write_dir (instruction[11:7]),    // rd
101         .write_data(wb_data),
102         .read_data1(rs1_data),
103         .read_data2(rs2_data)
104     );
105
106     //
            ------------------------------------------------------------------------
107     // ID/EX: Immediate Generation and Operand Select
108     //
            ------------------------------------------------------------------------
109
110     logic [2:0]            imm_sel;
111         logic [DATA_WIDTH-1:0] imm;
112
113     imm_gen imm_gen_i (
114         .instr   (instruction),
```

```
115         .imm_sel (imm_sel),
116         .imm_out (imm)
117     );
118
119     // ALU operand mux controls
120     logic [1:0]             op1_sel_pc;     // 00: use zero as operand 1, 01:
        use PC as operand 1, 10: use rs1 as operand 1
121     logic                   op2_sel_imm;    // 1: use imm, 0: use rs2 as
        operand 2
122
123     logic [DATA_WIDTH-1:0] alu_op1;               // ALU operand 1
124     logic [DATA_WIDTH-1:0] alu_op2;               // ALU operand 2
125     logic [DATA_WIDTH-1:0] zero_operand = 32'd0;  // Zero operand for LUI
        operation
126     // Operand1: PC vs rs1
127     mux_operand_1 alu_op1_mux_i (
128     .in1(zero_operand),   // Zero operand as operand 1
129     .in2(pc_q),           // PC as operand 1
130     .in3(rs1_data),       // rs1 as operand 1
131     .sel(op1_sel_pc),     // 00: use zero as operand 1, 01: use PC as operand
        1, 10: use rs1 as operand 1
132     .data_out(alu_op1)    // data_out as operand_1
133     );
134
135     // Operand2: imm vs rs2
136     mux #(.WIDTH(DATA_WIDTH)) alu_op2_mux_i (
137         .in1(imm),          // sel=1 -> imm
138         .in2(rs2_data),     // sel=0 -> rs2
139         .sel(op2_sel_imm),
140         .out(alu_op2)
141     );
142
143     //
        ------------------------------------------------------------------------

144     // EX: ALU and Branch Decision
145     //
        ------------------------------------------------------------------------

146
147     logic [3:0]             alu_ctrl;
148     logic [DATA_WIDTH-1:0] alu_result;
149
150     alu #(.OPERAND_WIDTH(DATA_WIDTH)) alu_i (
151         .operand1   (alu_op1),
152         .operand2   (alu_op2),
153         .alucontrol (alu_ctrl),
154         .alu_result (alu_result)
155     );
156
157     // For branches/jumps in this single-cycle design, branch_target is
        produced by ALU
158     // when operand1=PC and operand2=imm and alu_ctrl=ADD.
159     assign branch_target = alu_result;
160
161     // Comparator for TYPE-B conditions
```

```
    logic b_condition_rs1_rs2;

    branch #(.DATA_WIDTH_BRANCH(DATA_WIDTH)) branch_cmp_i (
        .rs_1           (rs1_data),
        .rs_2           (rs2_data),
        .opcode         (instruction[6:0]),
        .funct_3        (instruction[14:12]),
        .branch_taken   (b_condition_rs1_rs2)
    );

    //
        ------------------------------------------------------------------------

    // MEM: Data Memory (store/load datapath)
    //
        ------------------------------------------------------------------------


    logic                   dmem_we;
    logic [DATA_WIDTH-1:0] dmem_rdata;

    data_memory #(.DATA_WIDTH(DATA_WIDTH), .DEPTH(DEPTH)) data_mem_i (
        .clk     (clk),
        .wr_en   (dmem_we),
        .addr    (alu_result),    // byte address from ALU
        .data_in (rs2_data),      // store data
        .data_out(dmem_rdata)     // load data
    );
                                     Draft
    //
        ------------------------------------------------------------------------

    // WB: Write-back Mux (ALU vs MEM vs PC+4)
    //
        ------------------------------------------------------------------------


    logic [1:0] wb_sel;

    mux_3_to_1 wb_mux_i (
        .data_out_to_pc   (pc_plus_inc),  // PC+4 (link address)
        .alu_to_mem_addr  (alu_result),   // ALU result
        .data_out_to_mux  (dmem_rdata),   // memory read data
        .sel              (wb_sel),
        .data_out         (wb_data)
    );

    //
        ------------------------------------------------------------------------

    // Control Unit: Decode instruction -> drive control signals
    //
        ------------------------------------------------------------------------


    control_unit cu_i (
```

```
        .opcode             (instruction[6:0]),
        .funct_3            (instruction[14:12]),
        .funct_7            (instruction[31:25]),
        .zero               (b_condition_rs1_rs2),     // B condition

        .prf_wr_en          (rf_we),
        .cu_imm_sel         (imm_sel),
        .prf_pc_mux_ctrl    (op1_sel_pc),
        .prf_imm_mux_ctrl   (op2_sel_imm),
        .cu_alu_ctrl        (alu_ctrl),
        .cu_mem_out_mux_sel(wb_sel),
        .cu_data_mem_wr_en  (dmem_we),
        .cu_pc_add_sel      (pc_inc_sel),
        .branch_taken       (branch_taken)
    );

    assign instruction = instr_en ? driven_instr : mem_instruction;

endmodule
```

2.4 RSICV-32 Top Diagram

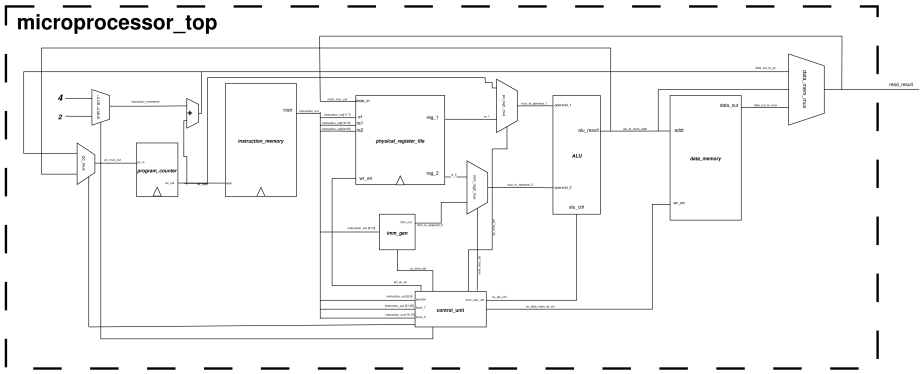

Figure 1: Architecture of Microprocessor.

Draft

3 Processor Specifications

3.1 Architectural Overview

The implemented processor is a 32-bit RV32I single-cycle RISC-V microprocessor. It executes one complete instruction per clock cycle, including instruction fetch, decode, execute, memory access (if required), and write-back.

The architecture is fully synchronous and operates under a single clock domain. All state elements (Program Counter, Register File, and memories) are updated on the rising edge of the clock.

The datapath width is 32 bits, and the processor supports the base integer instruction set defined by the RV32I specification.

3.2 Supported Instruction Types

The processor supports the following opcode groups:

- **R-Type** (0110011) – Register-to-register arithmetic and logical operations.
- **I-Type** (0010011) – Immediate arithmetic and logical operations.
- **L-Type** (0000011) – Load instructions.
- **S-Type** (0100011) – Store instructions.
- **B-Type** (1100011) – Conditional branch instructions.
- **U-Type LUI** (0110111) – Load Upper Immediate.
- **U-Type AUIPC** (0010111) – Add Upper Immediate to PC.
- **J-Type (JAL)** (1101111) – Jump and Link.

3.3 Supported Arithmetic and Logical Operations

For R-type and I-type instructions, the following `funct3`-based operations are supported:

- ADD / SUB
- SLL (Shift Left Logical)
- SLT (Set Less Than, signed)
- SLTU (Set Less Than, unsigned)
- XOR
- SRL / SRA (Shift Right Logical / Arithmetic)
- OR
- AND

3.4 Supported Branch Operations

The processor supports the following conditional branch instructions:

- BEQ (Branch if Equal)
- BNE (Branch if Not Equal)
- BLT (Branch if Less Than, signed)
- BGE (Branch if Greater or Equal, signed)
- BLTU (Branch if Less Than, unsigned)
- BGEU (Branch if Greater or Equal, unsigned)

3.5 Supported Load and Store Operations

Load instructions:

- LB (Load Byte, signed)
- LH (Load Halfword, signed)
- LW (Load Word)
- LBU (Load Byte, unsigned)
- LHU (Load Halfword, unsigned)

Store instructions:

- SB (Store Byte)
- SH (Store Halfword)
- SW (Store Word)

3.6 Control Flow and Program Counter Behavior

The Program Counter (PC) is byte-addressed and normally increments by 4 for sequential instruction execution. When a branch or jump condition is satisfied, the PC is redirected to the computed target address.

3.7 Behavior for Invalid or Unsupported Instructions

If the control unit receives an instruction with an unsupported or undefined opcode, the processor shall not stall or enter an undefined state.

Instead, the following behavior is expected:

- The instruction is treated as a NOP (No Operation).

- The PC continues to increment sequentially (PC + 4).
- No register write or memory write operation is performed.

In this implementation, the default NOP behavior corresponds to an `ADDI x0, x0, 0` instruction, which produces no architectural state change and ensures forward program progress.

This requirement guarantees forward progress and prevents deadlock conditions caused by invalid instruction decoding.

4 Verification Plan

4.1 Verification Objectives

The objective of this verification plan is to validate the functional correctness, control integrity, and architectural compliance of the RV32I single-cycle microprocessor.

Verification focuses on ensuring:

- Correct execution of all supported legal instructions.
- Proper Program Counter (PC) behavior under sequential and control-flow instructions.
- Correct register file behavior, including preservation of `x0 = 0`.
- Correct ALU computation for arithmetic and logical operations.
- Correct memory interface behavior for load and store instructions.
- Safe forward progress in the presence of illegal or unsupported instructions.

The verification strategy is assertion-driven and instruction-aware, meaning that properties are evaluated dynamically according to the decoded instruction type.

4.2 Test Plan Strategy

The processor is verified using the following stimulus categories:

1. Directed Legal Instruction Testing

The processor is stimulated with all supported legal instruction encodings. Each opcode and its associated `funct3` and `funct7` combinations are exercised.

The goal of this phase is to:

- Validate datapath correctness.
- Validate control signal generation.
- Verify write-back correctness.
- Confirm correct branch and jump redirection.

2. Illegal Instruction Testing

The processor is stimulated with unsupported or invalid opcode encodings.

Expected behavior:

- The processor must not stall or enter an undefined state.
- No unintended register or memory write shall occur.
- The PC shall continue sequential execution (PC + 4).

This test ensures architectural robustness and forward progress.

3. Constrained Random Legal Instruction Testing

Randomly generated legal instructions are applied to the processor.

The objective is to:

- Stress control logic transitions.
- Exercise diverse register and immediate combinations.
- Detect corner-case arithmetic or comparison errors.
- Validate cumulative execution behavior over extended instruction streams.

Random testing increases confidence beyond directed stimulus by exploring unexpected operand combinations.

4.3 Assertion-Based Verification Strategy

Verification is performed using SystemVerilog Assertions (SVA). Assertions are bound non-intrusively to the DUT and are evaluated for every decoded instruction.

Each assertion is conditionally activated based on the current instruction type. This ensures that properties are instruction-aware and context-sensitive.

The assertions are grouped as follows:

1. Global Architectural Properties

These properties must hold for all instructions:

- **PC alignment:** The Program Counter must always be word-aligned.
- **Sequential progression:** Non-branch instructions must advance PC by 4.
- **Register x0 invariant:** Register x0 must always remain zero.

2. I-Type Instruction Properties

For I-type arithmetic instructions:

- Immediate selection must correspond to I-format.
- Register write-enable must be asserted.
- Write-back data must equal the expected ALU computation.

Assertions validate:

- ADDI result correctness.
- SLTIU comparison behavior.
- XORI, ORI, ANDI bitwise correctness.

3. R-Type Instruction Properties

For register-to-register instructions:

- Correct arithmetic behavior (ADD, SUB).
- Correct shift operations (SLL, SRL, SRA).
- Correct signed and unsigned comparisons (SLT, SLTU).
- Correct bitwise logic (AND, OR, XOR). Draft

Each operation has a dedicated property that verifies write-back data against the expected ALU result.

4. Control-Flow Instruction Properties

For control instructions:

- JAL must redirect PC to the computed target.
- AUIPC must write PC + immediate.
- LUI must correctly place the immediate in the upper bits.
- Branch instructions must update PC conditionally based on comparison results.

Branch assertions explicitly verify both taken and not-taken paths.

5. Memory Instruction Properties

For memory operations:

- Store instructions must assert data memory write enable.
- Store address must equal RS1 + IMM.
- Load instructions must disable memory write.
- Load operations must write memory data back into the register file.

4.4 Verification Execution Model

Assertions are evaluated on every decoded instruction using an instruction-triggered property macro structure. This ensures:

- Deterministic checking per instruction.
- Immediate failure detection.
- Clear localization of functional errors.

This assertion-centric strategy provides cycle-accurate validation of datapath, control logic, and architectural invariants.

5 Functional Coverage

5.1 Coverage Objectives

In addition to assertion-based checking, functional coverage is collected to quantify how thoroughly the verification stimulus exercises the supported instruction space and key decode/execute combinations.

Coverage is used to confirm that:

- All supported opcode groups (R/I/L/S/B/LUI/AUIPC/JAL) are observed during simulation.
- The relevant `funct3` variants for I-type and R-type instructions are exercised.
- R-type `funct7` variants are exercised, specifically ensuring SUB and SRA decode paths are hit.
- Cross-coverage between `funct3` and `funct7` is achieved for R-type instructions.
- Illegal/unsupported instruction activity can be detected (either directly or through the NOP mapping behavior).

The resulting coverage reports are collected from simulation and documented at the end of this section using screenshots of the generated coverage summary.

5.2 Cover Properties

Cover properties are defined to confirm that each major instruction class is observed at least once. These covers are instruction-driven and trigger when the opcode matches the corresponding group.

```
// ------------------------------------------------------------------------
// COVERS
// ------------------------------------------------------------------------
`COV(it, cov_any_i_type, (`OPCODE == OPCODE_I_TYPE) |->, 1'b1)
`COV(it, cov_any_r_type, (`OPCODE == OPCODE_R_TYPE) |->, 1'b1)
`COV(it, cov_any_b_type, (`OPCODE == OPCODE_B_TYPE) |->, 1'b1)
`COV(it, cov_any_l_type, (`OPCODE == OPCODE_L_TYPE) |->, 1'b1)
`COV(it, cov_any_s_type, (`OPCODE == OPCODE_S_TYPE) |->, 1'b1)
`COV(it, cov_any_lui,    (`OPCODE == OPCODE_U_LUI)  |->, 1'b1)
`COV(it, cov_any_auipc,  (`OPCODE == OPCODE_U_AUIPC)|->, 1'b1)
`COV(it, cov_any_jal,    (`OPCODE == OPCODE_JAL_TYPE)|->, 1'b1)
```

5.3 Covergroups and Coverpoints

To obtain finer-grain insight beyond opcode-level observation, a covergroup is defined and sampled on every clock cycle after reset. This covergroup captures opcode distribution, instruction variants through `funct3` and `funct7`, and key cross-coverage for R-type instructions.

```
// ------------------------------------------------------------------------
// COVERGROUPS
// ------------------------------------------------------------------------

// Sample coverage on every cycle after reset
covergroup cg_instr @(posedge clk);
  option.per_instance = 1;

  // Cover that we ever saw a NOP on the instruction bus
  cp_nop: coverpoint (`OPCODE == 7'b1111111) iff (arst_n) {
    bins seen_nop = {1'b1};
  }

  // Basic opcode coverage
  cp_opcode: coverpoint `OPCODE iff (arst_n) {
    bins r_type = {OPCODE_R_TYPE};
    bins i_type = {OPCODE_I_TYPE};
    bins l_type = {OPCODE_L_TYPE};
    bins s_type = {OPCODE_S_TYPE};
    bins b_type = {OPCODE_B_TYPE};
    bins lui    = {OPCODE_U_LUI};
    bins auipc  = {OPCODE_U_AUIPC};
    bins jal    = {OPCODE_JAL_TYPE};
    // If illegal opcodes are mapped to NOP by the control logic, track them
       as "other"
    bins other  = default;
  }

  // I-type funct3 coverage (only meaningful for I-type)
  cp_funct3_i: coverpoint `FUNCT3 iff (arst_n && (`OPCODE == OPCODE_I_TYPE)) {
    bins addi  = {F3_ADD_SUB};
    bins slti  = {F3_SLT};
    bins sltiu = {F3_SLTU};
    bins xori  = {F3_XOR};
    bins ori   = {F3_OR};
    bins andi  = {F3_AND};
  }

  // R-type funct3 coverage (only meaningful for R-type)
  cp_funct3_r: coverpoint `FUNCT3 iff (arst_n && (`OPCODE == OPCODE_R_TYPE)) {
    bins add_sub = {F3_ADD_SUB};
    bins sll     = {F3_SLL};
    bins slt     = {F3_SLT};
    bins sltu    = {F3_SLTU};
    bins xor_    = {F3_XOR};
    bins srl_sra = {F3_SRL_SRA};
    bins or_     = {F3_OR};
    bins and_    = {F3_AND};
  }

```

```
50    // R-type funct7 coverage (SUB/SRA differentiation)
51    cp_funct7_r: coverpoint `FUNCT7 iff (arst_n && (`OPCODE == OPCODE_R_TYPE)) {
52      bins base = {7'b0000_000};
53      bins sub  = {7'b0100_000};
54    }
55
56    // Cross-coverage: funct3 x funct7 for R-type
57    cx_rtype: cross cp_funct3_r, cp_funct7_r iff (arst_n && (`OPCODE ==
          OPCODE_R_TYPE));
58
59  endgroup
60
61  cg_instr cg_instr_i;
62
63  initial begin
64    // Create covergroup instance
65    cg_instr_i = new();
66  end
```

5.4 Coverage Report Evidence

The final coverage results are obtained from the simulator coverage report. Screenshots of the coverage summary and detailed bins are included below.

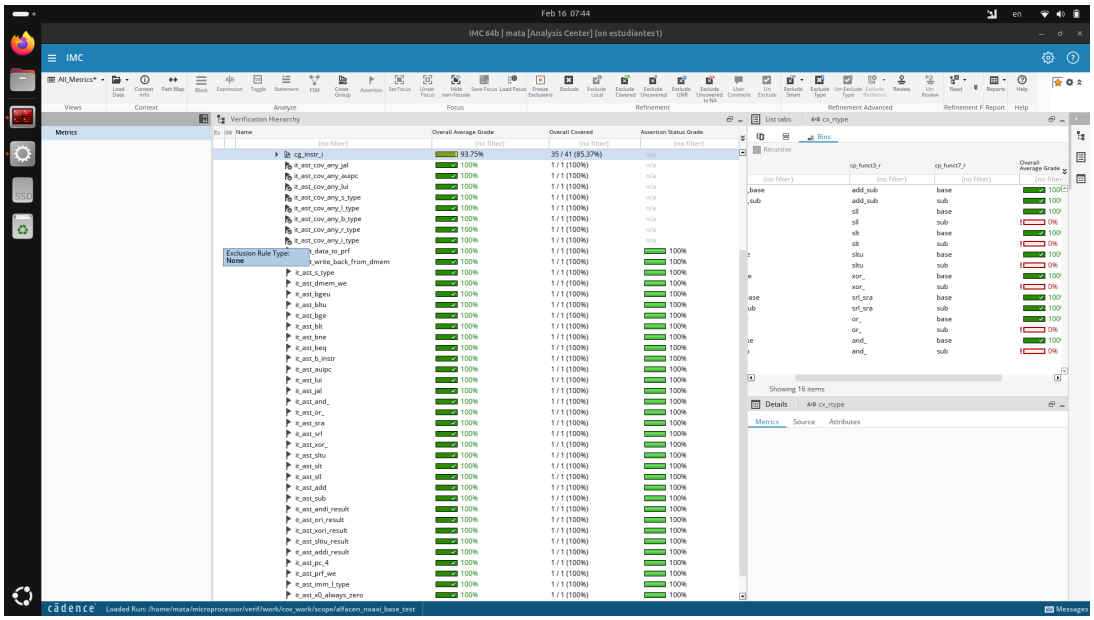

Figure 2: Functional coverage summary corresponding to opcode-level and instruction-variant coverage.

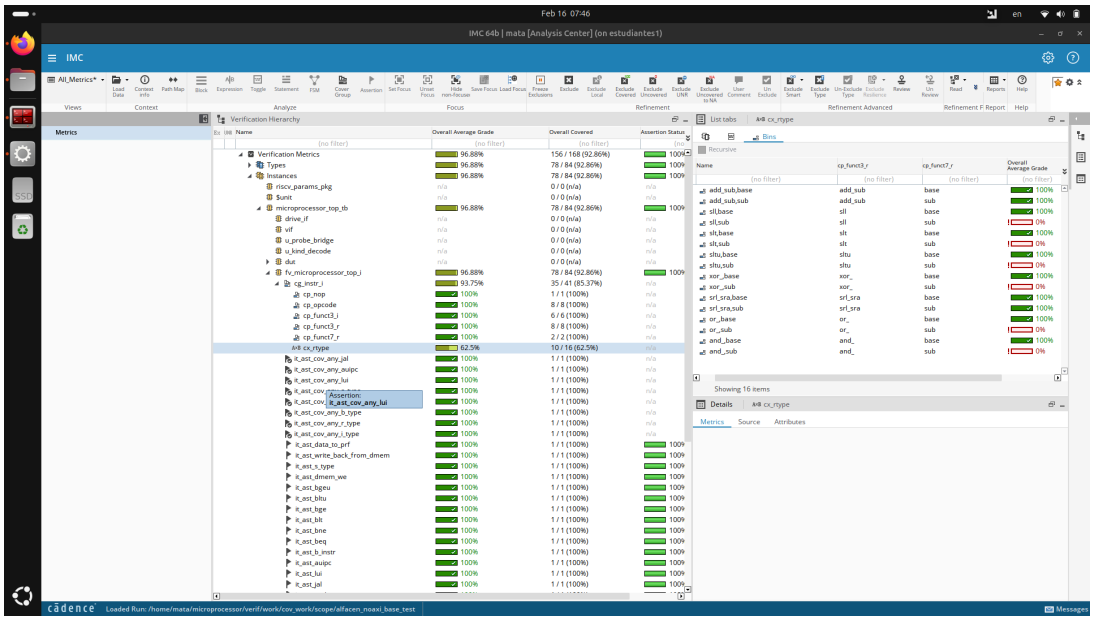

Figure 3: Detailed covergroup bin coverage including `funct3`, `funct7`, and defined cross coverage.

6 Appendix A: RTL Modules - Most Significative

6.1 control_unit - module

Draft

```
‘timescale 1ns / 1ps
//
    ----------------------------------------------------------------------------------

// Module: control_unit
// Description:
//   Combinational control unit for a single-cycle RV32I-style datapath.
//   Decodes {opcode, funct3, funct7} and generates control signals for:
//     - Register file write-back enable and write-back source selection
//     - Immediate format selection (imm_gen control)
//     - ALU operation selection
//     - Data memory write enable (stores)
//     - PC update decision (sequential vs branch/jump target)
//
// Supported instructions (current):
//   - I-type: ADDI, SLTI, SLTIU, XORI, ORI, ANDI
//   - R-type: ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND
//   - B-type: BEQ (branch decision uses input 'zero')
//   - J-type: JAL (unconditional)
//
// Assumptions:
//   - 'zero' is asserted when rs1 == rs2 (BEQ compare evaluated externally).
//   - Control encodings (IMM_*, ALU_*, mux selects, opcodes) are defined in
    defines.svh.
//   - PC-target computation (PC + imm) and PC muxing are implemented outside
    this module.
```

```
//
// Notes:
//   - Default assignments implement the common I-type ALU-immediate path.
//     Opcode decoding below overrides only what differs per instruction class.
//   - Assertions for illegal/unsupported encodings can be added in a later stage.
//
//------------------------------------------------------------------------------

import riscv_params_pkg::*;
module control_unit(
    input  logic [6:0]  opcode,             // instr[6:0]
    input  logic [2:0]  funct_3,            // instr[14:12]
    input  logic [6:0]  funct_7,            // instr[31:25]
    input  logic        zero,               // 1 when rs1 == rs2 (BEQ condition)
    output logic        prf_wr_en,           // register file write enable
    output logic [2:0]  cu_imm_sel,          // immediate format select (imm_gen)
    output logic [1:0]  prf_pc_mux_ctrl,     // ALU operand1 select (rs1 vs PC)
    output logic        prf_imm_mux_ctrl,    // ALU operand2 select (rs2 vs imm)
    output logic [3:0]  cu_alu_ctrl,         // ALU operation select
    output logic [1:0]  cu_mem_out_mux_sel,  // write-back mux select (to PRF)
    output logic        cu_data_mem_wr_en,   // data memory write enable (stores)
    output logic        cu_pc_add_sel,       // PC increment select (+4 now; +2 reserved)
    output logic        branch_taken         // PC redirect request (branch/jump)
);

    always_comb begin
        //------------------------------------------------------------------------------

        // Defaults: I-type ALU-immediate behavior
        //   - rd write enabled
        //   - ALU computes rs1 + imm
        //   - PC advances sequentially (+4)
        // Opcode decode below overrides these defaults as required.
        //------------------------------------------------------------------------------

        prf_wr_en          = 1'b1;
        cu_data_mem_wr_en  = 1'b0;
        cu_imm_sel         = IMM_I;
        prf_pc_mux_ctrl    = 2'b10;        // operand1 = rs1
        prf_imm_mux_ctrl   = 1'b1;        // operand2 = imm
        cu_pc_add_sel      = 1'b0;        // +4 in RV32; +2 reserved for future compressed support
        cu_mem_out_mux_sel = ALU_TO_PRF;  // write-back = ALU result
        cu_alu_ctrl        = ALU_ADD;
        branch_taken       = 1'b0;        // default: no PC redirect
```

```
63
64          unique case (opcode)
65
66              // I-type ALU-immediate ops: rd <- rs1 op imm
67              OPCODE_I_TYPE: begin
68                  unique case (funct_3)
69                      F3_ADD_SUB:   cu_alu_ctrl = ALU_ADD;
70                      F3_SLL:   cu_alu_ctrl = ALU_SLT;
71                      F3_SLTU:  cu_alu_ctrl = ALU_SLTU;
72                      F3_XOR:   cu_alu_ctrl = ALU_XOR;
73                      F3_OR:    cu_alu_ctrl = ALU_OR;
74                      F3_AND:   cu_alu_ctrl = ALU_AND;
75                      default:cu_alu_ctrl = ALU_ADD;
76                  endcase
77              end
78
79              // B-type branch: no write-back; PC redirected when branch_taken=1
80              OPCODE_B_TYPE: begin
81                  prf_wr_en       = 1'b0;   // branches do not write rd
82                  cu_imm_sel      = IMM_B;  // branch offset immediate
83                  prf_pc_mux_ctrl = 2'b01;   // operand1 = rs1 (as defined by
                        your datapath for target calc)
84                  cu_alu_ctrl   = ALU_ADD; // used for target computation
                        elsewhere (PC + imm path)
85                  branch_taken  = zero;    // redirect request when condition is
                         true
86                  unique case (funct_3)
87                      // BEQ: take branch when rs1 == rs2 (zero=1)
88                      F3_BEQ: ;        Draft
89                      // BNE: take branch when rs1 != rs2 (zero=1)
90                      F3_BNE: ;
91                      // BLT: take branch when rs1 < rs2 (zero=1)
92                      F3_BLT: ;
93                      // BGE: take branch when rs1 >= rs2 (zero=1)
94                      F3_BGE: ;
95                      // BLTU: take branch when rs1 < rs2 (zero=1)
96                      F3_BLTU: ;
97                      // BGEU: take branch when rs1 >= rs2 (zero=1)
98                      F3_BGEU: ;
99                      default: begin
100                         cu_alu_ctrl  = ALU_ADD;
101                         branch_taken = 1'b0;
102                     end
103                 endcase
104             end
105
106             // R-type ALU-register ops: rd <- rs1 op rs2
107             OPCODE_R_TYPE: begin
108                 prf_imm_mux_ctrl = 1'b0; // operand2 = rs2 (not immediate)
109
110                 // funct7 selects ADD/SUB and SRL/SRA families in RV32I
111                 if (funct_7 == 7'b0000_000) begin
112                     unique case (funct_3)
113                         F3_ADD_SUB:   cu_alu_ctrl = ALU_ADD;
114                         F3_SLL:   cu_alu_ctrl = ALU_SLL;
115                         F3_SLT:   cu_alu_ctrl = ALU_SLT;
```

```
                    F3_SLTU:  cu_alu_ctrl = ALU_SLTU;
                    F3_XOR:  cu_alu_ctrl = ALU_XOR;
                    F3_SRL_SRA:   cu_alu_ctrl = ALU_SRL;
                    F3_OR:   cu_alu_ctrl = ALU_OR;
                    F3_AND:  cu_alu_ctrl = ALU_AND;
                    default:cu_alu_ctrl = ALU_ADD;
                endcase
            end else if (funct_7 == 7'b0100_000) begin
                unique case (funct_3)
                    F3_ADD_SUB:   cu_alu_ctrl = ALU_SUB;
                    F3_SRL_SRA:   cu_alu_ctrl = ALU_SRA;
                    default:cu_alu_ctrl = ALU_ADD;
                endcase
            end else begin
                cu_alu_ctrl = ALU_ADD; // unsupported funct7 -> safe
                    default
            end
        end

        // JAL: rd <- PC+4, PC <- PC + imm (unconditional redirect)
        OPCODE_JAL_TYPE: begin
            cu_imm_sel         = IMM_J;               // jump offset
                immediate
            prf_pc_mux_ctrl    = 2'b01;                // operand1 = PC (
                for target calc path)
            cu_mem_out_mux_sel = INSTRUCTION_TO_PRF;  // write-back
                selects PC+4 (per your datapath naming)
            cu_alu_ctrl        = ALU_ADD;
            branch_taken       = 1'b1;                // unconditional
                redirect
        end
        // load-word
        OPCODE_L_TYPE: begin
            cu_mem_out_mux_sel = DATA_OUT_TO_PRF;
            unique case (funct_3)
                F3_LB:     ;
                F3_LH:     ;
                F3_LW:     ;
                F3_LBU:    ;
                F3_LHU:    ;
                default: prf_wr_en = 1'b1;
            endcase
        end
        // write-word
        OPCODE_S_TYPE: begin
            prf_wr_en = 1'b0;
            cu_mem_out_mux_sel = ALU_TO_PRF;
            cu_imm_sel         = IMM_S;
            cu_data_mem_wr_en  = 1'b1;
            unique case (funct_3)
                F3_SB:     ;
                F3_SH:     ;
                F3_SW:     ;
                default: prf_wr_en = 1'b0;
            endcase
        end
```

```
            // LUI operation
            OPCODE_U_LUI: begin
                prf_pc_mux_ctrl    = 2'b00;                 // operand1 = PC
                cu_imm_sel         = IMM_U;                // Extend the width
                    of imm
            end
            // AUIPC operation
            OPCODE_U_AUIPC: begin
                prf_pc_mux_ctrl    = 2'b01;                 // operand1 = PC
                cu_imm_sel         = IMM_U;                // Extend the width
                    of imm
            end
            // Default: NOP
            default: begin
                prf_wr_en    = 1'b0;
                cu_alu_ctrl  = ALU_ADD;
                branch_taken = 1'b0;
            end
        endcase
    end

endmodule
```

6.2 program_counter - module

```
`timescale 1ns / 1ps
//                                    Draft
    ----------------------------------------------------------------------------------


// Module: program_counter
// Description:
//   Program counter (PC) register. Captures the next PC value on each rising
//   clock edge and provides the current PC to the instruction fetch path.
//
// Assumptions:
//    - arst_n is an active-low asynchronous reset.
//
// Notes:
//   - Reset initializes PC to 0x0000_0000.
//   - This block is purely sequential storage; PC update selection is handled
//     elsewhere in the datapath/control.
//
    ----------------------------------------------------------------------------------


module program_counter #(
        parameter DATA_WIDTH = 32,
        parameter [DATA_WIDTH-1:0] RESET_PC = '0
)(
        input logic clk,
        input logic arst_n,
        input logic [DATA_WIDTH - 1:0] pc_in,
        output logic [DATA_WIDTH - 1:0] pc_out
);
    //Secuential logic
    always_ff @(posedge clk, negedge arst_n) begin
```

```
        if (!arst_n) begin
            pc_out <= RESET_PC; //Reset PC to boot address
        end else  begin
            pc_out <= pc_in;
        end
    end
endmodule
```

6.3 transmitter - module

```

`timescale 1ns / 1ps
//
    //////////////////////////////////////////////////////////////////////////////////

// Company:
// Engineer: Manuel Enrique Rivera Acosta
//
// Create Date: 10/28/2025 12:38:11 PM
// Design Name:
// Module Name: transmitter
// Project Name:
//
    //////////////////////////////////////////////////////////////////////////////////


                                    Draft
module transmitter #(parameter BYTE_WIDTH = 8)(
    input logic clk, // clock signal
    input logic arst_n, // asynchronous reset
    input logic [BYTE_WIDTH - 1 : 0] data_in, // DATA_IN TO BE SENT TO THE
        RECEIVER MODULE BIT BY BIT THROUGH THE TX
    input logic tick, // TICK COMING FROM THE BAUD RATE GENERATOR
    input logic tx_start, // TX_START SIGNAL TO BEGIN THE BYTE TRANSMISSION
    output logic tx, // SERIAL OUTPUT BIT TO TRANSMIT INTO THE RECEIVER RX
        INPUT SIGNAL
    output logic tx_done // DONE SIGNAL INDICATING THAT THE BYTE HAS BEEN SENT
);

localparam BIT_SAMPLING = 15; // localparam for the oversampling counter of
    the uart x16

logic [BYTE_WIDTH - 1 : 0] shifted_data; // temporal register to store the
    data shifted to the tx
logic [BYTE_WIDTH - 1 : 0] shifted_data_next; // next-state temporal register
    shifted_data

logic [4:0] nbits; // variable to count the number of bits that has been sent
logic [4:0] nbits_next; // next-state nbits variable

logic [4:0] oversampling_count; // oversampling counter for the uart x16
logic [4:0] oversampling_count_next; // next-state oversampling counter

logic tx_done_next; // next-state tx_done signal (transmission done signal)
logic tx_next;  // next-state tx signal (serial output bit)
```

```
37
38 typedef enum logic [2:0]  {IDLE = 3'b000, START = 3'b001, DATA = 3'b010, STOP
       = 3'b011} state_type;
39
40 state_type state_reg, state_next;
41
42 always_ff @(posedge clk or negedge arst_n) begin
43     if(!arst_n) begin
44         state_reg <= IDLE;
45     end else begin
46         state_reg <= state_next;
47         nbits <= nbits_next;
48         oversampling_count <= oversampling_count_next;
49         shifted_data <= shifted_data_next;
50         tx_done <= tx_done_next;
51         tx <= tx_next;
52     end
53 end
54
55 always_comb begin
56         state_next = state_reg;
57         nbits_next = nbits;
58         oversampling_count_next = oversampling_count;
59         shifted_data_next = shifted_data;
60         tx_done_next = tx_done;
61         tx_next = tx;
62
63     case(state_reg)
64                                       Draft
65         IDLE: begin
66             tx_done_next = 1'b0;
67             nbits_next = '0;
68             tx_next = 1'b1;
69             oversampling_count_next = '0;
70             if(tx_start) begin
71                 shifted_data_next = data_in;
72                 tx_next = 1'b0;
73                 state_next = START;
74             end
75         end
76
77         START: begin
78             if(tick) begin
79                 if(oversampling_count == BIT_SAMPLING) begin
80                     oversampling_count_next = '0;
81                     state_next = DATA;
82                 end else begin
83                     oversampling_count_next = oversampling_count + 1;
84                 end
85             end
86         end
87
88         DATA: begin
89             tx_next = shifted_data[0];
90             if(tick) begin
91                 if(oversampling_count == BIT_SAMPLING) begin
```

```
                    shifted_data_next = {1'b0, shifted_data[7:1]};
                    oversampling_count_next = '0;
                    if(nbits == BYTE_WIDTH - 1) begin
                        state_next = STOP;
                    end else begin
                        nbits_next = nbits + 1'b1;
                    end
                end else begin
                    oversampling_count_next = oversampling_count + 1;
                end
            end
        end

        STOP: begin
            tx_next = 1'b1;
            if(tick) begin
                if(oversampling_count == BIT_SAMPLING) begin
                    state_next = IDLE;
                    tx_done_next = 1'b1;
                end else begin
                    oversampling_count_next = oversampling_count + 1;
                    state_next = STOP;
                end
            end
        end
        default: begin
                state_next = state_reg;
        end
    endcase                                 Draft
end

endmodule
```

6.4 imm_gen - module

```
`timescale 1ns / 1ps
import riscv_params_pkg::*;
//
    ----------------------------------------------------------------------------


// Module: imm_gen
// Description:
//   RISC-V RV32I immediate generator. Extracts and sign/zero-extends the
//   immediate field from a 32-bit instruction according to imm_sel.
//
// Assumptions:
//   - instr is a valid RV32I instruction word.
//   - imm_sel is provided by the control unit and selects the immediate
    format:
//       IMM_I: I-type   (sign-extended 12-bit immediate)
//       IMM_S: S-type   (sign-extended 12-bit store immediate)
//       IMM_B: B-type   (sign-extended branch offset, LSB forced to 0)
//       IMM_U: U-type   (upper immediate, lower 12 bits are 0)
//       IMM_J: J-type   (sign-extended jump offset, LSB forced to 0)
//
```

```
// Notes:
//    - Branch/jump immediates are instruction-aligned; therefore bit[0] is 0.
//    - This block performs field extraction/extension only. Target address
//      computation (PC + imm) is performed elsewhere.
//
    ------------------------------------------------------------------------------

module imm_gen(
    input  logic [31:0] instr,
    input  logic [2:0]  imm_sel,
    output logic [31:0] imm_out
);

    always_comb begin
        imm_out = '0; // default

        unique case (imm_sel)

            // I-type immediate: imm[11:0] = instr[31:20], sign-extended to 32
               b
            IMM_I: imm_out = {{20{instr[31]}}, instr[31:20]};

            // S-type immediate: imm[11:5]=instr[31:25], imm[4:0]=instr[11:7],
                sign-extended
            IMM_S: imm_out = {{20{instr[31]}}, instr[31:25], instr[11:7]};

            // B-type immediate (branch offset):
            // imm[12]=instr[31], imm[11]=instr[7], imm[10:5]=instr[30:25],
               imm[4:1]=instr[11:8], imm[0]=0
            IMM_B: imm_out = {{19{instr[31]}}, instr[31], instr[7], instr
               [30:25], instr[11:8], 1'b0};

            // U-type immediate: imm[31:12]=instr[31:12], lower 12 bits are
               zero
            IMM_U: imm_out = {instr[31:12], 12'b0};

            // J-type immediate (jump offset):
            // imm[20]=instr[31], imm[19:12]=instr[19:12], imm[11]=instr[20],
               imm[10:1]=instr[30:21], imm[0]=0
            IMM_J: imm_out = {{11{instr[31]}}, instr[31], instr[19:12], instr
               [20], instr[30:21], 1'b0};

            default: imm_out = '0;
        endcase
    end

endmodule
```


6.5 microprocessor_top_tb - module

```
`timescale 1ns/1ps

//import riscv_params_pkg::*;

module microprocessor_top_tb;
```

```
6
7   //
        ------------------------------------------------------------------------
8   // Clock / Reset
9   //
        ------------------------------------------------------------------------
10  bit clk;
11  bit arst_n;
12
13  initial clk = 1'b0;
14  always #5ns clk = ~clk;
15
16  initial begin
17    arst_n = 1'b0;
18    repeat (3) @(posedge clk);
19    arst_n = 1'b1;
20  end
21
22  //
        ------------------------------------------------------------------------
23  // Interfaces
24  //
        ------------------------------------------------------------------------
25  instr_drive_if #(DATA_WIDTH) drive_if (clk);
26  microprocessor_if #(DATA_WIDTH, DIR_WIDTH) vif (clk);
27  microprocessor_probe_bridge u_probe_bridge (.vif(vif));
28
29  riscv32_rand_instruction tr;
30
31  instr_kind_decode_for_waves u_kind_decode (.vif(vif));
32
33  //
        ------------------------------------------------------------------------
34  // DUT
35  //
        ------------------------------------------------------------------------
36  microprocessor_top dut (
37    .clk          (clk),
38    .arst_n       (arst_n),
39    .instr_en     (drive_if.instr_en),
40    .driven_instr (drive_if.driven_instr)
41  );
42
43  //
        ------------------------------------------------------------------------
44  // CHECKER
45  //
        ------------------------------------------------------------------------
```

```
46  fv_microprocessor_top #(
47    .DATA_W(DATA_WIDTH),
48    .DIR_W(DIR_WIDTH)
49  ) fv_microprocessor_top_i (
50    .clk   (clk),
51    .arst_n(arst_n),
52    .vif   (vif)
53  );
54
55  //
        ============================================================================

56  // Initialize
57  //
        ============================================================================

58  initial begin : init_state
59    for (int i = 1; i < 32; i++) begin
60      dut.prf_i.prf[i] = '0;
61    end
62
63    for (int j = 0; j < DEPTH; j++) begin
64      dut.data_mem_i.mem[j] = '0;
65    end
66  end
67
68  //
        ============================================================================
                                        Draft
69  // TEST_PROGRAM
70  //
        ============================================================================

71  initial begin : test_program
72    tr = new();
73
74    // ------------------------------------------------------------
75    // Wait until reset is released
76    // ------------------------------------------------------------
77    wait (arst_n == 1'b1);
78    repeat (2) @(posedge clk);
79
80    // ------------------------------------------------------------
81    // 1) Directed sweep: hit every legal instruction at least once
82    // ------------------------------------------------------------
83    drive_if.drive_all_legal_once();
84
85    // Optional idle cycle
86    @(posedge clk);
87
88    // ------------------------------------------------------------
89    // 2) Illegal opcodes: must not be decoded
90    // ------------------------------------------------------------
91    drive_if.drive_illegal_nop_test(500);
92
93    // Optional idle cycle
```

```
    @(posedge clk);

    // ------------------------------------------------------------
    // 3) Random test
    // ------------------------------------------------------------
    repeat (100_000) begin
      if (!tr.randomize()) $fatal("Randomize failed");
      drive_if.drive_item(tr);
    end

    @(posedge clk);
    $finish;
  end

/*  initial begin : shm_db
    // Initial block to open shared memory and probe signals
    $shm_open("shm_db");
    $shm_probe("ASMTR");
  end
*/
endmodule
```

6.6 fv_microprocessor_top - module

```
// fv/fv_microprocessor_top.sv
`timescale 1ns/1ps
//`include "property_defines.svh"

//
    ------------------------------------------------------------------------------

// Module: fv_microprocessor_top
// Description:
//   Top-level SVA checker using microprocessor_if signals.
//   - Keeps the original assertion text and naming.
//   - Uses vif.* as the signal source.
//
    ------------------------------------------------------------------------------


module fv_microprocessor_top #(
  parameter int DATA_W = 32,
  parameter int DIR_W  = 5
)(
  input  logic clk,
  input  logic arst_n,
  microprocessor_if.monitor vif
);


  //
      ------------------------------------------------------------------------

  // ASSERTIONS
```

```
25  //
        ------------------------------------------------------------------------
26
27  //
        ========================================================================
28  //  PC AND PRF
29  `AST (it, pc_aligned,
30       1'b1 |->, (`PC_Q[1:0] == 2'b00)
31  )
32
33  `AST(it, next_instr,
34      (`OPCODE inside{ OPCODE_R_TYPE, OPCODE_I_TYPE, OPCODE_U_LUI,
            OPCODE_U_AUIPC, OPCODE_L_TYPE, OPCODE_S_TYPE}) |->,
35      (`PC_NEXT == `PC_Q + 3'b100)
36  )
37
38  `AST (it, x0_always_zero,
39       1'b1 |->, (`X0 == '0)
40  )
41
42  //
        ========================================================================
43  //  TYPE_I INSTRUCTIONS
44  `AST (it, imm_I_type,
45       (`OPCODE == OPCODE_I_TYPE) |->, (`IMM_SEL == IMM_I)
46  )                                    Draft
47
48  `AST(it, prf_we,
49      (`OPCODE == OPCODE_I_TYPE) |->, (`RF_WE == 1'b1)
50  )
51
52  `AST(it, pc_4,
53      (`OPCODE == OPCODE_I_TYPE) |->, (`PC_NEXT == `PC_Q + 3'b100)
54  )
55
56  // ========================================================
57  // I-type: ADDI, SLTI, SLTIU, XORI, ORI, ANDI
58  // ========================================================
59
60  `AST(it, addi_result,
61    ((`OPCODE == OPCODE_I_TYPE) && (`FUNCT3 == F3_ADD_SUB)) |->,
62    (`WB_DATA == (`ALU_OP1 + `ALU_OP2))
63  )
64
65 /*  `AST(it, slti_result,
66     ((`OPCODE == OPCODE_I_TYPE) && (`FUNCT3 == F3_SLT)) |->,
67     (`WB_DATA ==
68     ( $signed(`ALU_OP1) < $signed(`ALU_OP2) ))
69  )*/
70
71
72  `AST(it, sltiu_result,
73    ((`OPCODE == OPCODE_I_TYPE) && (`FUNCT3 == F3_SLTU)) |->,
```

```
74      (`WB_DATA == ((`ALU_OP1 < `ALU_OP2) ? 32'd1 : 32'd0))
75    )
76
77    `AST(it, xori_result,
78      ((`OPCODE == OPCODE_I_TYPE) && (`FUNCT3 == F3_XOR)) |->,
79      (`WB_DATA == (`ALU_OP1 ^ `ALU_OP2))
80    )
81
82    `AST(it, ori_result,
83      ((`OPCODE == OPCODE_I_TYPE) && (`FUNCT3 == F3_OR)) |->,
84      (`WB_DATA == (`ALU_OP1 | `ALU_OP2))
85    )
86
87    `AST(it, andi_result,
88      ((`OPCODE == OPCODE_I_TYPE) && (`FUNCT3 == F3_AND)) |->,
89      (`WB_DATA == (`ALU_OP1 & `ALU_OP2))
90    )
91
92    //  ============================================================
93    //  R-type: ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND
94    //  ============================================================
95
96    `AST(it, sub,
97        ((`OPCODE == OPCODE_R_TYPE) && (`FUNCT3 == F3_ADD_SUB) && (`FUNCT7 == 7'
            b0100_000)) |->,
98        (`WB_DATA == (`RS1_DATA - `RS2_DATA))
99    )
100
101   `AST(it, add,                          Draft
102       ((`OPCODE == OPCODE_R_TYPE) && (`FUNCT3 == F3_ADD_SUB) && (`FUNCT7 == 7'
            b0000_000)) |->,
103       (`WB_DATA == (`RS1_DATA + `RS2_DATA))
104   )
105
106   `AST(it, sll,
107       ((`OPCODE == OPCODE_R_TYPE) && (`FUNCT3 == F3_SLL) && (`FUNCT7 == 7'
            b0000_000)) |->,
108       (`WB_DATA == (`RS1_DATA << `RS2_DATA[4:0]))
109   )
110
111   `AST(it, slt,
112       ((`OPCODE == OPCODE_R_TYPE) && (`FUNCT3 == F3_SLT) && (`FUNCT7 == 7'
            b0000_000)) |->,
113       ((`WB_DATA) == ($signed(`RS1_DATA) < $signed(`RS2_DATA) ? 32'd1 : 32'd0)
            )
114   )
115
116   `AST(it, sltu,
117       ((`OPCODE == OPCODE_R_TYPE) && (`FUNCT3 == F3_SLTU) && (`FUNCT7 == 7'
            b0000_000)) |->,
118       (`WB_DATA == (`RS1_DATA < `RS2_DATA ? 32'd1 : 32'd0) )
119   )
120
121   `AST(it, xor_,
122       ((`OPCODE == OPCODE_R_TYPE) && (`FUNCT3 == F3_XOR) && (`FUNCT7 == 7'
            b0000_000)) |->,
```

```
123         (`WB_DATA == (`RS1_DATA ^ `RS2_DATA))
124     )
125
126     `AST(it, srl,
127         ((`OPCODE == OPCODE_R_TYPE) && (`FUNCT3 == F3_SRL_SRA) && (`FUNCT7 == 7'
                b0000_000)) |->,
128         (`WB_DATA == (`RS1_DATA >> `RS2_DATA[4:0]))
129     )
130
131     `AST(it, sra,
132         ((`OPCODE == OPCODE_R_TYPE) && (`FUNCT3 == F3_SRL_SRA) && (`FUNCT7 == 7'
                b0100_000)) |->,
133         ($signed(`WB_DATA) == ($signed(`RS1_DATA) >>> `RS2_DATA[4:0]) )
134     )
135
136     `AST(it, or_,
137         ((`OPCODE == OPCODE_R_TYPE) && (`FUNCT3 == F3_OR) && (`FUNCT7 == 7'
                b0000_000)) |->,
138         (`WB_DATA == (`RS1_DATA | `RS2_DATA))
139     )
140
141     `AST(it, and_,
142         ((`OPCODE == OPCODE_R_TYPE) && (`FUNCT3 == F3_AND) && (`FUNCT7 == 7'
                b0000_000)) |->,
143         (`WB_DATA == (`RS1_DATA & `RS2_DATA))
144     )
145
146     //
            ================================Draft=========================================
147     //  JAL (JUMP AND LINK) INSTRUCTION
148     //
            ================================================================================
149     `AST(it, jal,
150         (`OPCODE == OPCODE_JAL_TYPE) |->,
151         (`PC_NEXT == (`PC_Q + `ALU_OP2))
152     )
153
154     //
            ================================================================================
155     // LUI INSTRUCTION
156     //
            ================================================================================
157     `AST (it, lui,
158          (`OPCODE == OPCODE_U_LUI) |->,
159          (`WB_DATA == (32'd0 + `IMM))
160     )
161
162     //
            ================================================================================
163     //  AUIPC INSTRUCTION
```

```
164  //
         ================================================================================
165  `AST (it, auipc,
166       (`OPCODE == OPCODE_U_AUIPC) |->,
167       (`WB_DATA == (`PC_Q + `IMM))
168  )
169
170  //
         ================================================================================
171  //  TYPE B INSTRUCTIONS
172  //
         ================================================================================
173
174  `AST(it, b_instr,
175      (`OPCODE == OPCODE_B_TYPE) |->,
176      ((`BR_TAKEN) ? (`PC_NEXT == `PC_Q + `IMM): (`PC_NEXT == `PC_Q + 3'b100))
177  )
178
179  `AST(it, beq,
180      ((`OPCODE == OPCODE_B_TYPE) && (`FUNCT3 == F3_BEQ) ) |->,
181      ((`RS1_DATA == `RS2_DATA) ? (`PC_NEXT == `PC_Q + `IMM): (`PC_NEXT == `
            PC_Q + 3'b100))
182  )
183
184  `AST(it, bne,
185      ((`OPCODE == OPCODE_B_TYPE) && (`FUNCT3 == F3_BNE) ) |->,
186      ((`RS1_DATA != `RS2_DATA) ? (`PC_NEXT == `PC_Q + `IMM): (`PC_NEXT == `
            PC_Q + 3'b100))
187  )
188
189  `AST(it, blt,
190      ((`OPCODE == OPCODE_B_TYPE) && (`FUNCT3 == F3_BLT) ) |->,
191      (($signed(`RS1_DATA) < $signed(`RS2_DATA)) ?  (`PC_NEXT == `PC_Q + `IMM)
            : (`PC_NEXT == `PC_Q + 3'b100))
192  )
193
194  `AST(it, bge,
195      ((`OPCODE == OPCODE_B_TYPE) && (`FUNCT3 == F3_BGE) ) |->,
196      (($signed(`RS1_DATA) >= $signed(`RS2_DATA)) ? (`PC_NEXT == `PC_Q + `IMM)
            : (`PC_NEXT == `PC_Q + 3'b100))
197  )
198
199  `AST(it, bltu,
200      ((`OPCODE == OPCODE_B_TYPE) && (`FUNCT3 == F3_BLTU) ) |->,
201      ((`RS1_DATA < `RS2_DATA) ? (`PC_NEXT == `PC_Q + `IMM): (`PC_NEXT == `
            PC_Q + 3'b100))
202  )
203
204  `AST(it, bgeu,
205      ((`OPCODE == OPCODE_B_TYPE) && (`FUNCT3 == F3_BGEU) ) |->,
206      ((`RS1_DATA >= `RS2_DATA) ? (`PC_NEXT == `PC_Q + `IMM): (`PC_NEXT == `
            PC_Q + 3'b100))
207  )
```

```
208
209 //
    ================================================================================
210 //  TYPE S INSTRUCTIONS
211 //
    ================================================================================
212 `AST(it,dmem_we,
213     (`OPCODE == OPCODE_S_TYPE) |->,
214     (`DMEM_WE == 1'b1)
215 )
216
217 `AST(it, s_type,
218     (`OPCODE == OPCODE_S_TYPE) |->,
219     ((`DMEM_ADDR == `RS1_DATA + `IMM) && (`DMEM_WDATA == `RS2_DATA))
220 )
221
222 //
    ================================================================================
223 //  TYPE L INSTRUCTIONS
224 //
    ================================================================================
225 `AST(it,write_back_from_dmem,
226     (`OPCODE == OPCODE_L_TYPE) |->,
227     ((`DMEM_WE == 1'b0) && (`RF_WE == 1'b1))
228 )                                   Draft
229
230 `AST(it, data_to_prf,
231     (`OPCODE == OPCODE_L_TYPE) |->,
232     (`DMEM_ADDR == `RS1_DATA + `IMM )
233 )
234
235 //
    --------------------------------------------------------------------------
236 // COVERS
237 //
    --------------------------------------------------------------------------
238 `COV(it, cov_any_i_type, (`OPCODE == OPCODE_I_TYPE) |->, 1'b1)
239 `COV(it, cov_any_r_type, (`OPCODE == OPCODE_R_TYPE) |->, 1'b1)
240 `COV(it, cov_any_b_type, (`OPCODE == OPCODE_B_TYPE) |->, 1'b1)
241 `COV(it, cov_any_l_type, (`OPCODE == OPCODE_L_TYPE) |->, 1'b1)
242 `COV(it, cov_any_s_type, (`OPCODE == OPCODE_S_TYPE) |->, 1'b1)
243 `COV(it, cov_any_lui,    (`OPCODE == OPCODE_U_LUI)  |->, 1'b1)
244 `COV(it, cov_any_auipc,  (`OPCODE == OPCODE_U_AUIPC)|->, 1'b1)
245 `COV(it, cov_any_jal,    (`OPCODE == OPCODE_JAL_TYPE)|->, 1'b1)
246
247 //
    --------------------------------------------------------------------------
248 // COVERGROUPS
```

```
249  //
         ------------------------------------------------------------------------
250
251
252  // Sample coverage on every cycle after reset
253  covergroup cg_instr @(posedge clk);
254    option.per_instance = 1;
255
256    // Cover that we ever saw a NOP on the instruction bus
257    cp_nop: coverpoint (`OPCODE == 7'b1111111) iff (arst_n) {
258      bins seen_nop = {1'b1};
259    }
260
261    // Basic opcode coverage (expand with more bins later)
262    cp_opcode: coverpoint `OPCODE iff (arst_n) {
263      bins r_type = {OPCODE_R_TYPE};
264      bins i_type = {OPCODE_I_TYPE};
265      bins l_type = {OPCODE_L_TYPE};
266      bins s_type = {OPCODE_S_TYPE};
267      bins b_type = {OPCODE_B_TYPE};
268      bins lui    = {OPCODE_U_LUI};
269      bins auipc  = {OPCODE_U_AUIPC};
270      bins jal    = {OPCODE_JAL_TYPE};
271      // If your CU collapses illegal opcodes into NOP, you can still track "
             others"
272      bins other  = default;
273    }
                                       Draft
274
275    // Funct3 only makes sense for some opcodes; use conditional bins
276    cp_funct3_i: coverpoint `FUNCT3 iff (arst_n && (`OPCODE == OPCODE_I_TYPE))
           {
277      bins addi  = {F3_ADD_SUB};
278      bins slti  = {F3_SLT};
279      bins sltiu = {F3_SLTU};
280      bins xori  = {F3_XOR};
281      bins ori   = {F3_OR};
282      bins andi  = {F3_AND};
283    }
284
285    cp_funct3_r: coverpoint `FUNCT3 iff (arst_n && (`OPCODE == OPCODE_R_TYPE))
           {
286      bins add_sub = {F3_ADD_SUB};
287      bins sll     = {F3_SLL};
288      bins slt     = {F3_SLT};
289      bins sltu    = {F3_SLTU};
290      bins xor_    = {F3_XOR};
291      bins srl_sra = {F3_SRL_SRA};
292      bins or_     = {F3_OR};
293      bins and_    = {F3_AND};
294    }
295
296    // Funct7 only meaningful for R-type (SUB/SRA)
297    cp_funct7_r: coverpoint `FUNCT7 iff (arst_n && (`OPCODE == OPCODE_R_TYPE))
           {
298      bins base = {7'b0000_000};
```

```
299       bins sub  = {7'b0100_000};
300     }
301
302     // R-type funct3 x funct7 (ensures SUB/SRA paths get hit)
303     cx_rtype: cross cp_funct3_r, cp_funct7_r iff (arst_n && (`OPCODE ==
            OPCODE_R_TYPE));
304
305   endgroup
306
307   cg_instr cg_instr_i;
308
309   initial begin
310     // Create covergroup instance
311     cg_instr_i = new();
312   end
313
314 endmodule
```